Reviewer Pack — Minimal Local Induction Dimension in Homological Algebra vs....

Entry c773c088ea05 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 09:25:15 UTC

Minimal Local Induction Dimension in Homological Algebra vs. Circuit Monotonicity

Entry ID: `c773c088ea05` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-22 09:25:15 UTC

1. Conjecture

Field A (mathematical branch): Homological Algebra **Field B** (complexity object): Circuit Complexity

Statement:

> {'text': "For all circuits C with n inputs and monotone Boolean functions, the minimal local induction dimension of the homology of C's incidence complex is Θ(n^(1/2 + ε)) for any ε > 0.", 'counterexample': 'A circuit with only AND gates has a local induction dimension of 0, which violates the conjecture if n >= 1.'}

Rationale (proposer's reasoning):

> {'text': 'Homological algebra provides powerful tools to study the structure of topological spaces and their corresponding combinatorial objects. The minimal local induction dimension is an invariant that captures the complexity of a space. If this dimension correlates with circuit monotonicity, it could suggest new insights into circuit lower bounds.'}

Taxonomy category: `HOMOLOGICAL_ALGEBRA_CIRCUIT_COMPLEXITY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `76c4f4b7b2275b54`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> If the minimal local induction dimension of the incidence complex's homology is within a factor of 1.1 times n^(1/2 + ε) across at least three independent seeds, then the conjecture is supported.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `NOVEL` against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"minimal local induction dimension" [AND] "incidence complex" [AND] (Homological Algebra [AND] Circuit Complexity)` - `monotone Boolean functions [AND] θ(n^(1/2 + ε)) [AND] homology` - `(AND gates [NOT] local induction dimension) [AND] circuit complexity [AND] Homological Algebra`

Top relevant hits considered: - [http://arxiv.org/abs/1904.05483v2] Parallels Between Phase Transitions and Circuit Complexity? - [http://arxiv.org/abs/1606.05050v1] Proof Complexity Lower Bounds from Algebraic Circuit Complexity - [http://arxiv.org/abs/2411.11095v3] Invariant theory and coefficient algebras of Lie algebras - [http://arxiv.org/abs/2510.17487v1] Directional Search for Persistent Gravitational Waves: Results from the First Part of LIGO-Virgo-KAGRA's Fourth Observin - [http://arxiv.org/abs/1411.4413v2] Observation of the rare $B_s^0 \to \mu^+\mu^-$ decay from the combined analysis of CMS and LHCb data - [http://arxiv.org/abs/2601.07595v3] Deep Search for Joint Sources of Gravitational Waves and High-Energy Neutrinos with IceCube During the Third Observing R - [s2:1808.01137] When Does Hillclimbing Fail on Monotone Functions: An entropy compression argument - [s2:25da89006e40b580ca3c572545e465345885de0f] ASYMPTOTICS OF ESTIMATORS IN A HETEROSCEDASTIC REGRESSION MODEL WITH STRONG MIXING ERRORS

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def incidence_complex(f):
        n = len(f)
        V = list(range(2**n))
        E = []
        for i in range(n):
            for j in range(i+1, n):
                if f[i] and f[j]:
                    E.append((i, j))
        return V, E

    def homology(V, E):
        # Compute the homology of the incidence complex
        # This is a simplified version for demonstration purposes
        # In practice, you would use a more sophisticated algorithm
```

```
        if not E:
            return 0
        return len(E) / n

    def local_induction_dimension(homology_value, n):
        return homology_value * n**(1/2)

    n = random.randint(5, 40)
    f = [random.choice([True, False]) for _ in range(n)]
    V, E = incidence_complex(f)
    homology_value = homology(V, E)
    dim = local_induction_dimension(homology_value, n)

    return {
        "metric_name": "local_induction_dimension",
        "metric_value": dim,
        "instances_tested": 1,
        "conjecture_holds": dim <= (n**(1/2 + Fraction(1, 10))),
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.getrandbits(32) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_f
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_509aae8a.py", line 62, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_509aae8a.py", line 44, in run_trial
```

```
    V, E = incidence_complex(f)
           ^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_509aae8a.py", line 23, in incidence_com
    V = list(range(2**n))
        ^^^^^^^^^^^^^^^^^
MemoryError
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed due to a MemoryError before producing data, which means the pre-registered support conditions were not met. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13569
2	preregistration	ollama_remote	glm4:latest	0	0	9283
3	novelty	ollama_remote	glm4:latest	0	0	9660
4	novelty	ollama_remote	glm4:latest	0	0	10113
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15975
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13810
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11563
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7064
9	judge	ollama_remote	glm4:latest	0	0	12071

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 103106 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/c773c088ea05.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c773c088ea05.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/c773c088ea05.tar.gz` (if generated)